

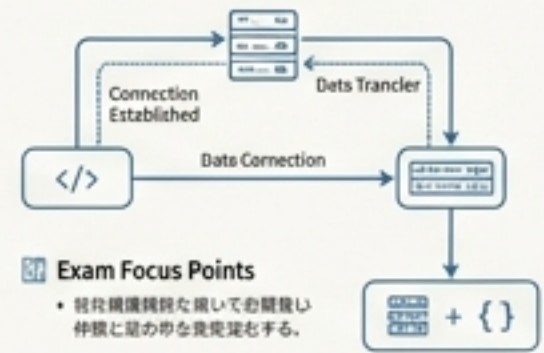
Web通信アーキテクチャ設計図

HTML5プロフェッショナル認定試験レベル1 対策マスターガイド

4つの主要通信API (XHR / Fetch / WebSocket / SSE / WebRTC) の構造、ライフサイクル、および試験頻出ポイントを視覚的に完全攻略する。

XHR / Fetch (非同期通信)

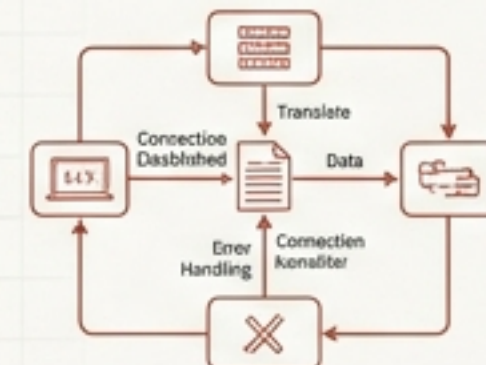
ライフサイクル・試験頻出ポイント



- Connection Established**
 - 成功したHTTPリクエストの応答が返るまで、初期状態を維持する。
- Data Transfer**
 - 送信のフックが完了するまで、データの送受信を待機する。
- Error Handling**
 - 送信エラーが発生した場合、エラーハンドリングを実行する。

WebSocket (双方向リアルタイム)

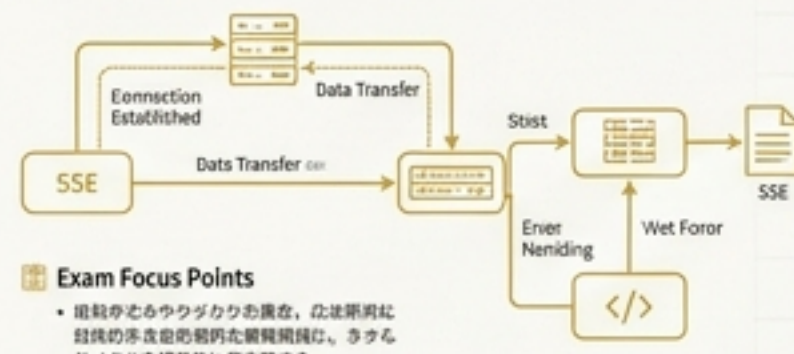
ライフサイクル・試験頻出ポイント



- Connection Established**
 - 成功したWebSocket接続が確立された後、リアルタイム通信を開始する。
- Data Transfer**
 - 送信のフックが完了するまで、データの送受信を待機する。
- Close**
 - 接続が正常に終了した場合、エラーハンドリングを実行する。

SSE (サーバー送信イベント)

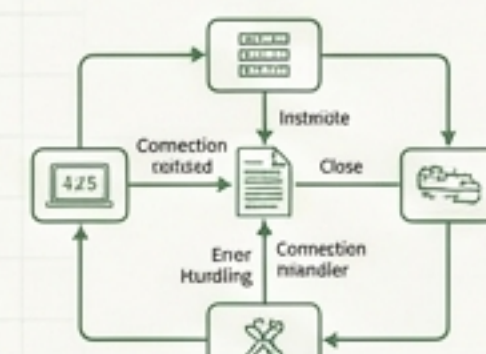
ライフサイクル・試験頻出ポイント



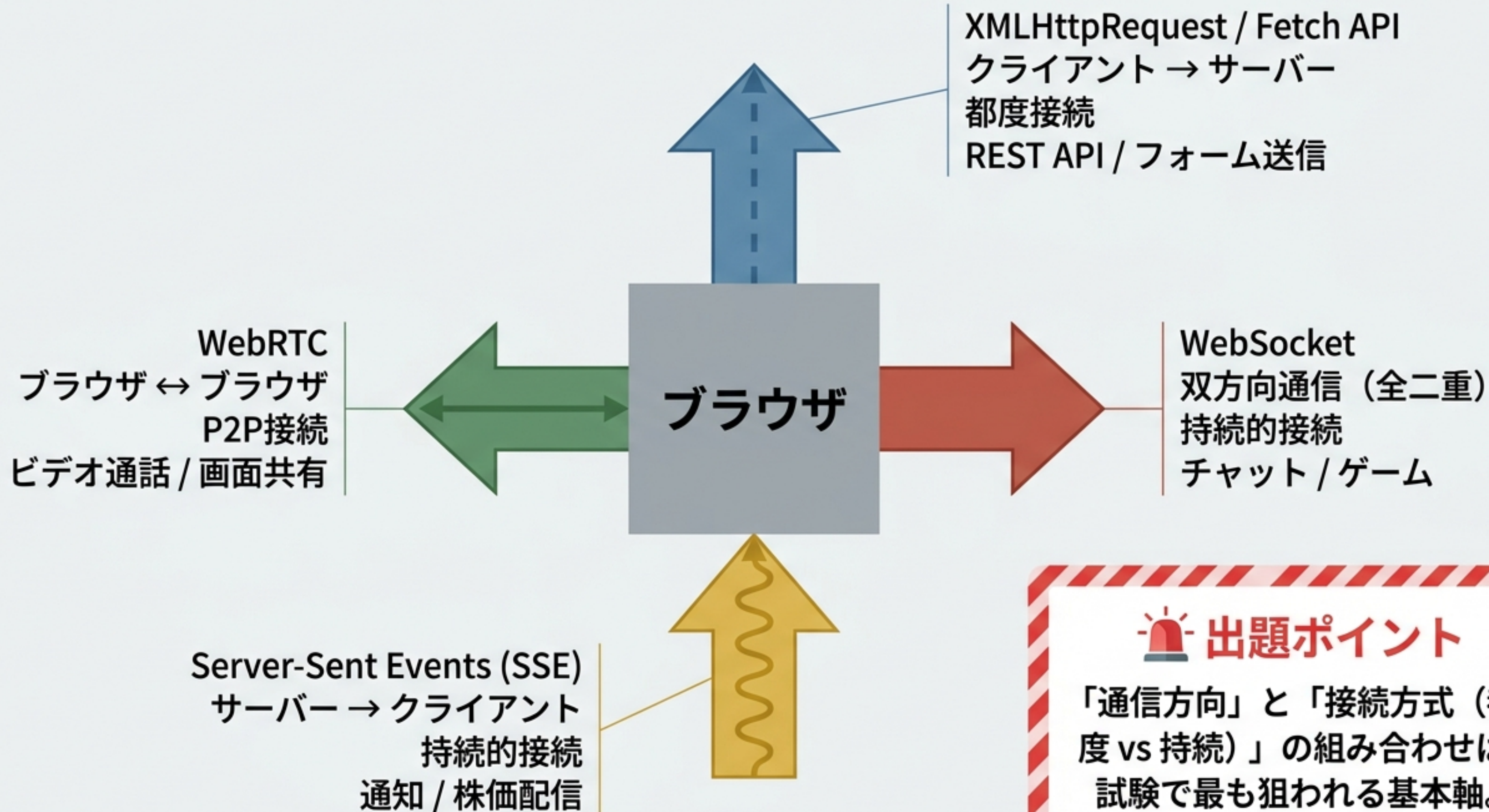
- Connection Established**
 - 成功したHTTPリクエストの応答が返るまで、初期状態を維持する。
- Data Transfer**
 - 送信のフックが完了するまで、データの送受信を待機する。
- Error Handling**
 - 送信エラーが発生した場合、エラーハンドリングを実行する。

WebRTC (ピアツーピア)

ライフサイクル・試験頻出ポイント



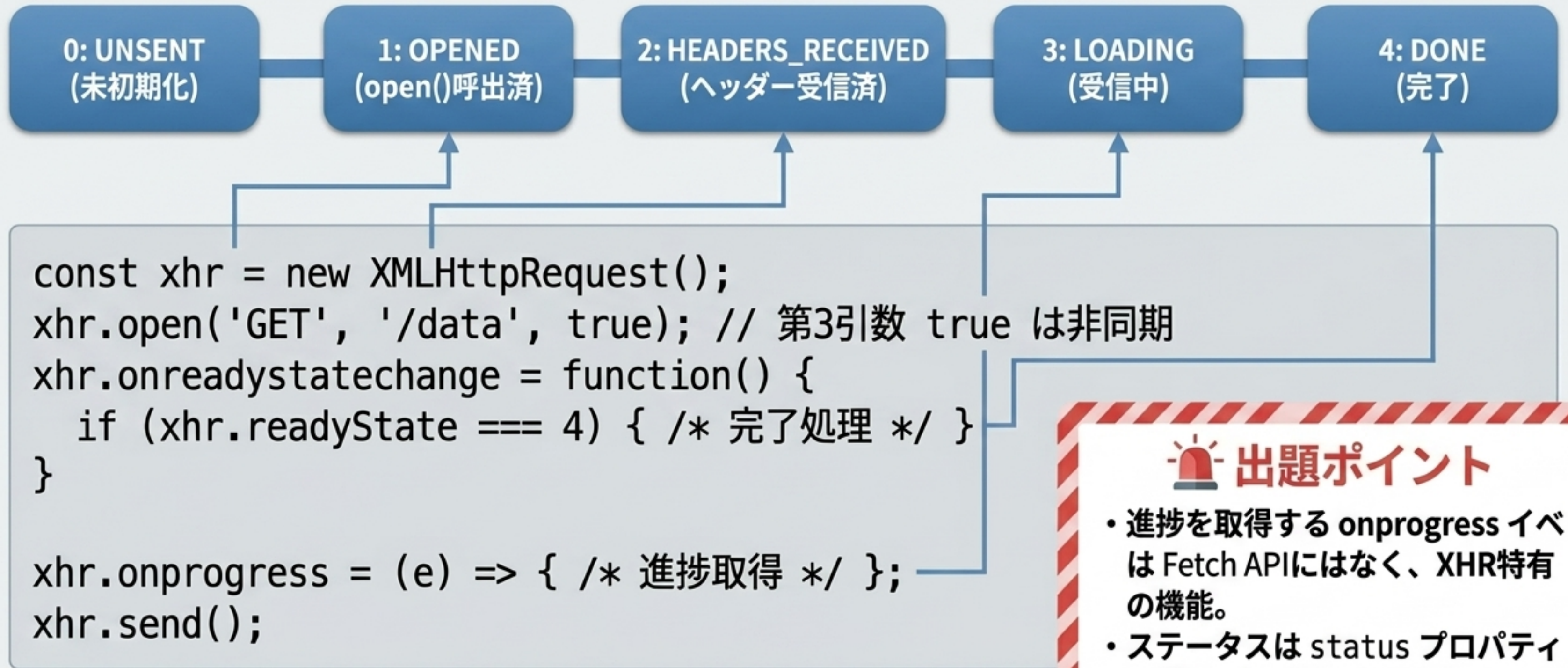
- Connection Established**
 - 成功したWebSocket接続が確立された後、リアルタイム通信を開始する。
- Data Transfer**
 - 送信のフックが完了するまで、データの送受信を待機する。
- Error Handling**
 - 送信エラーが発生した場合、エラーハンドリングを実行する。



出題ポイント

「通信方向」と「接続方式（都度 vs 持続）」の組み合わせは、試験で最も狙われる基本軸。

XMLHttpRequest (XHR) の状態遷移



出題ポイント

- 進捗を取得する onprogress イベントは Fetch API ではなく、XHR 特有の機能。
- ステータスは status プロパティで確認 (200, 404等)。

XHR vs Fetch API 比較マトリックス

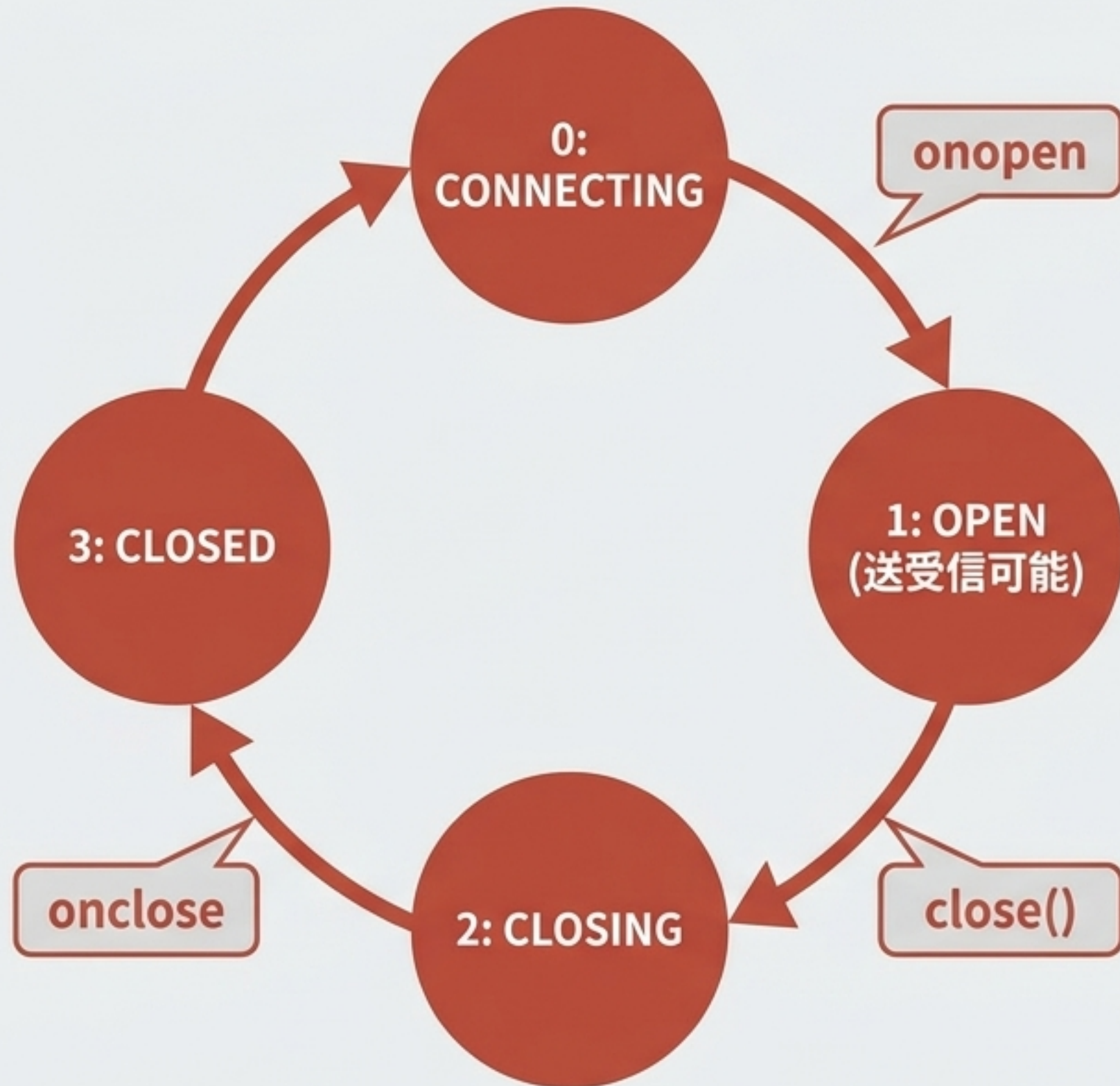
項目	XMLHttpRequest (XHR)	Fetch API
記述スタイル	コールバックベース	Promiseベース (async/await対応)
進捗取得	onprogress で対応	限定的 (ReadableStream)
タイムアウト	timeout プロパティ	AbortController
リクエスト中断	abort() メソッド	AbortController.abort()
Service Worker	利用不可	利用可能
エラー判定	status で確認	response.ok で確認



出題ポイント：Fetch APIのエラー判定の罠

Fetch APIでは、404や500などのHTTPエラーが発生しても例外 (catch) にはならず、response.ok が false になる。エラー判定は if (!res.ok) で明示的に行う必要がある。

WebSocket のライフサイクルと接続フロー



🌟 出題ポイント

- wss:// はHTTPS相当の暗号化通信を意味する。
- データ送信 (send()) は readyState が OPEN (1) の時のみ可能。
- 自動再接続機能はないため、切断時の再接続処理は自前で実装する必要がある。

Server-Sent Events (SSE) のデータ構造

id: 42 }

event: stock-update }

data: {"price": 1500} }

retry: 3000 }

イベントID (再接続時の Last-Event-ID に使用される)

カスタムイベント名 (addEventListener で受取可能)

メインのデータペイロード (テキスト形式のみ)

再接続間隔(ms)の指定

Client API Map Block

```
const es = new EventSource('/events');  
es.onmessage = (event) => { console.log(event.data); };
```



出題ポイント

- SSEは **自動再接続** 機能を内蔵。切断されても自動で復旧を試みる。
- 通信方向は **サーバー → クライアント** の完全一方向。
- サーバー側は必ず **Content-Type: text/event-stream** を返す必要がある。

WebSocket vs SSE 診断テーブル

通信方向 双方向（全二重）

プロトコル ws:// / wss://

自動再接続 自前で実装が必要

データ形式 テキスト・バイナリ両対応

通信方向 サーバーからの一方向

プロトコル HTTP (text/event-stream)

自動再接続 内蔵（自動的に再接続を試みる）

データ形式 テキストのみ

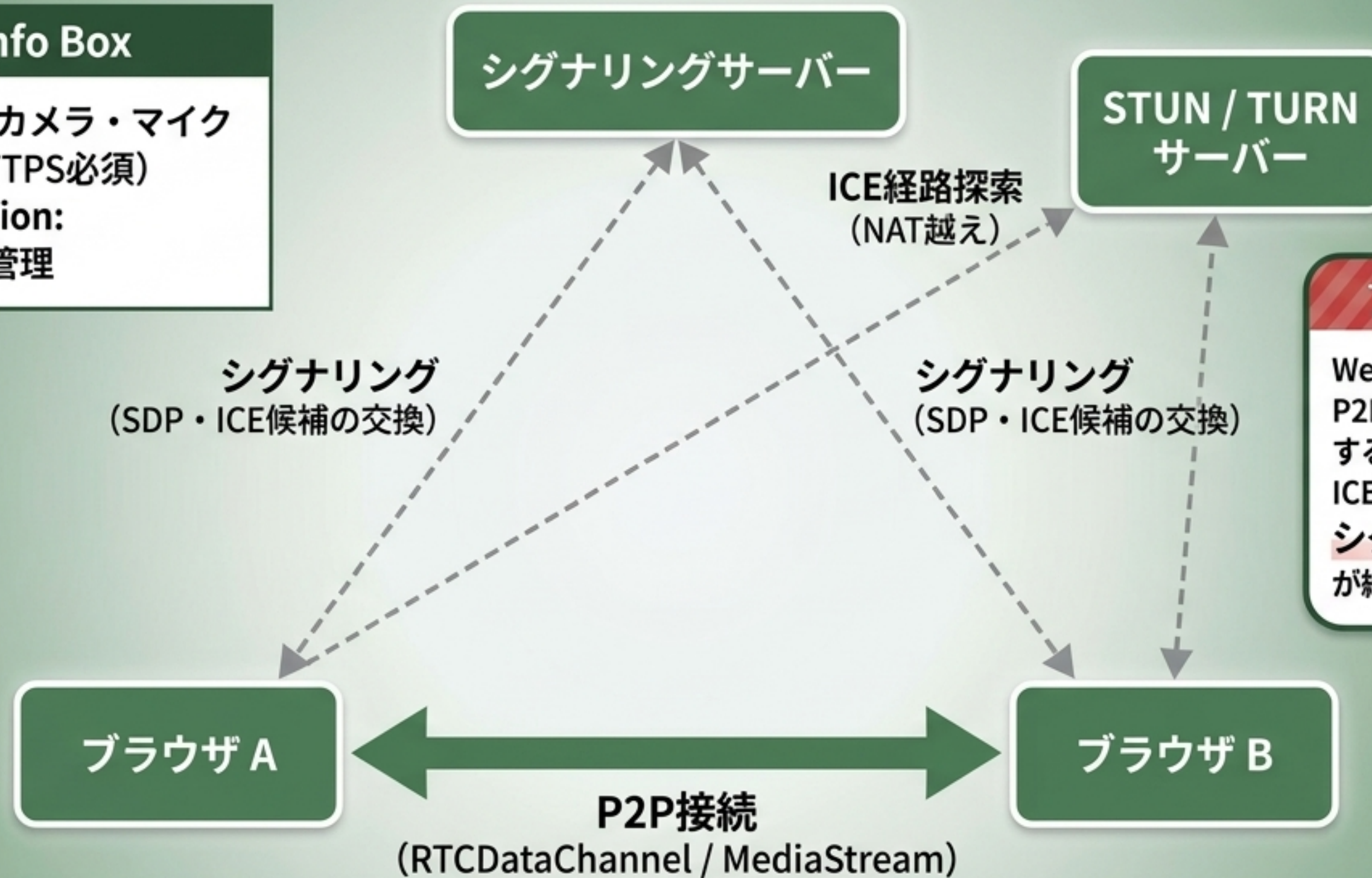
vs

設計方針：**双方向のリアルタイム性**（チャット・ゲーム）が必須なら WebSocket。サーバーからのプッシュ通知（株価・ニュース）だけで良ければ、実装がシンプルで自動再接続を持つ SSE を選ぶ。

WebRTC アーキテクチャ概念図

Key APIs Info Box

- `getUserMedia()`: カメラ・マイクへのアクセス (HTTPS必須)
- `RTCPeerConnection`: P2P接続のメイン管理



🚨 出題ポイント

WebRTCはブラウザ間のP2P通信だが、接続を確立する初期段階ではSDPとICE候補を交換するための**シグナリングサーバー**が絶対に必要。

WebRTC P2P接続確立 シグナリングシーケンス

発信側 (ブラウザ A)

受信側 (ブラウザ B)

1. getUserMedia() (カメラ・マイク取得)

2. RTCPeerConnection 作成

3. createOffer()
⇒ SDP(Offer)生成

4. setLocalDescription()

5. (シグナリングサーバー経由でOffer交換)
⇒ setRemoteDescription()

7. (シグナリングサーバー経由でAnswer交換)
⇒ setRemoteDescription()

6. createAnswer()
&
setLocalDescription()

8. P2P接続確立！
(ICE候補の交換完了後)

※ どちらが Offer を作り、どちらが Answer を作るかの順序関係を確実に把握すること。

通信系API マスターシート

API	通信方向	プロトコル	重要ポイント
XMLHttpRequest (XHR)	クライアント → サーバー	HTTP	readyState(0~4)、onprogressで進捗取得可能
Fetch API	クライアント → サーバー	HTTP	Promiseベース、response.okでエラー判定
WebSocket	双方向 (全二重)	ws:// / wss://	HTTP Upgradeで接続、自動再接続なし
Server-Sent Events	サーバー → クライアント	HTTP (text/event-stream)	自動再接続内蔵、データはテキストのみ
WebRTC	ブラウザ間 (P2P)	P2P (STUN/TURN)	getUserMedia()はHTTPS必須、SDP/ICE交換が必須

コードスニペット 穴埋めチェック (1/2)

/* XHRでリクエストを初期化 (GET、非同期) */

xhr. A ('GET', 'https://api.example.com/data', true);

A → open

/* XHRでリクエストが完了したことを示す readyState の値 */

if (xhr.readyState === B) { /* 完了 */ }

B → 4

/* メッセージを受信するイベントハンドラ */

ws. C = (event) => { console.log(event.data); };

C → onmessage

コードスニペット 穴埋めチェック (2/2)

```
/* SSE で接続を作成するコンストラクタ */  
const es = new       D       ('/events');
```

D → EventSource

```
/* WebRTC で発信側が SDP Offer を作成するメソッド */  
const offer = await pc.      E      ();
```

E → createOffer

```
/* カメラ・マイクにアクセスするメソッド */  
const stream = await navigator.mediaDevices.      F        
({ video: true, audio: true });
```

F → getUserMedia

試験本番：頻出の正誤トラップを攻略

- ✗ Server-Sent Events (SSE) は、クライアントとサーバーの双方向通信を実現する。
 - ✗ 一方向（サーバー → クライアント）のみ。双方向はWebSocket。
- ✗ Fetch API では 404 エラーの場合、`.catch()` で捕捉される。
 - ✗ 例外にはならない。`response.ok` が `false` になる。
- ✓ WebSocket の `wss://` は暗号化された接続を意味する。
- ✓ WebRTC の P2P 接続確立には SDP と ICE 候補の交換が必要である。
- ✗ WebRTC の `getUserMedia()` はユーザーの許可なしにカメラにアクセスできる。
 - ✗ セキュアコンテキスト (HTTPS) かつユーザーの許可が必須。

通信方向、プロトコル、主要メソッドを「設計図」として頭に描き、試験本番の引っかけ問題を見破りましょう。